# Exploratory Testing for Charter I

## Session Participants

Kevin He 261167637 — kevin.he2@mail.mcgill.ca (Session lead)

Eric Deng 261138815 — eric.deng@mail.mcgill.ca (Session participant)

## Section 1: Session Summary

### 1.1: Session Charter

Identify capabilities and areas of potential instability of the "rest api todo list manager". Identify documented and undocumented "rest api todo list manager" capabilities. For each capability, create a script or small program to demonstrate the capability. Exercise each capability identified with data typical to the intended use of the application.

Session focus: Explore /todos and /todos/:id with valid, invalid, JSON, XML and malformed data to discover capabilities and instabilities.

### 1.2: Version

Release version 1.5.5 of the Todo List Manager, running with profile version=3 (default data).

### 1.3: Set Up

Ubuntu 24.04 with Java 21. Requests were issued with curl from the session script so that every request can be replayed exactly; responses were captured in the session log. No Postman collection was used.

### 1.4: Build

Download from https://github.com/eviltester/thingifier/releases/tag/1.5.5 — runTodoManagerRestAPI-1.5.5.jar, started with: java -jar runTodoManagerRestAPI-1.5.5.jar (API and documentation at http://localhost:4567 and http://localhost:4567/docs).

## Section 2: Session Notes

### 2.1: Session Date and Time

October 14, 2026. Duration: 45 minutes (timeboxed). Requests are numbered by step in Table 2 in the exact order they were run; the raw log preserves that order (curl logs do not carry per-request wall-clock times).

### 2.2: Exploration Notes

Table 1: Legend for request colours

| Legend | |
|---|---|
| | As Expected |
| | Bug |
| | To Explore |

Table 2: Session Notes taken during Exploratory Testing

| Time | Endpoint Tested | Response | Reference File |
|---|---|---|---|
| Setup | **Run the application through the jar file**<br>java -jar runTodoManagerRestAPI-1.5.5.jar | Application available at http://localhost:4567; default (profile version=3) data present. | runTodoManagerRestAPI-1.5.5.jar (course-provided; not included in this package) |
| Step 1 | **Test API GET request**<br>/todos<br>Documented? DOCUMENTED<br>Expected: 200 with the list of all todos | **Status 200: OK**<br>Observed: HTTP 200 — {"todos":[{"id":"2","title":"file paperwork","doneStatus":"false","description":"","tasksof":[{"id":"1"}]},{"id":"1","title":"scan paperwork","doneStatus":"false","description":"","tasksof":[{"id":"1"}],"categories":[{"id":"1"}]}]<br>Result: **As Expected**<br>Side note: Default data: todos 1-2 | logs/exploratory_session1_todos.log<br>exploratory/session1_todos.sh |
| Step 2 | **Test API POST request**<br>/todos body: {"title":"buy milk"}<br>Documented? DOCUMENTED<br>Expected: 201 created; defaults doneStatus=false, description="" | **Status 201: Created**<br>Observed: HTTP 201 — {"id":"399","title":"buy milk","doneStatus":"false","description":""}<br>Result: **As Expected** | logs/exploratory_session1_todos.log<br>exploratory/session1_todos.sh |

| Time | Endpoint Tested | Response | Reference File |
|---|---|---|---|
| Step 3 | **Test API POST request**<br>/todos body: {"title":"full","doneStatus":true,"description":"all fields"}<br>Documented? DOCUMENTED<br>Expected: 201 created with all fields stored | **Status 201: Created**<br>Observed: HTTP 201 — {"id":"400","title":"full","doneStatus":"true","description":"all fields"}<br>Result: **As Expected** | logs/exploratory_session1_todos.log exploratory/session1_todos.sh |
| Step 4 | **Test API POST request**<br>/todos body: {}<br>Documented? DOCUMENTED<br>Expected: 400 "title : field is mandatory" | **Status 400: Bad Request**<br>Observed: HTTP 400 — {"errorMessages":["title : field is mandatory"]}<br>Result: **As Expected** | logs/exploratory_session1_todos.log exploratory/session1_todos.sh |
| Step 5 | **Test API POST request**<br>/todos body: {"title":"x","doneStatus":"maybe"}<br>Documented? DOCUMENTED<br>Expected: 400: doneStatus should be BOOLEAN | **Status 400: Bad Request**<br>Observed: HTTP 400 — {"errorMessages":["Failed Validation: doneStatus should be BOOLEAN"]}<br>Result: **As Expected**<br>Side note: See BUG-06 (typing) | logs/exploratory_session1_todos.log exploratory/session1_todos.sh |
| Step 6 | **Test API POST request**<br>/todos body: {"title":"x","id":99}<br>Documented? DOCUMENTED<br>Expected: 400: not allowed to create with id | **Status 400: Bad Request**<br>Observed: HTTP 400 — {"errorMessages":["Invalid Creation: Failed Validation: Not allowed to create with id"]}<br>Result: **As Expected** | logs/exploratory_session1_todos.log exploratory/session1_todos.sh |
| Step 7 | **Test API POST request**<br>/todos body: {"title":"x","colour":"red"}<br>Documented? DOCUMENTED<br>Expected: 400: could not find field colour | **Status 400: Bad Request**<br>Observed: HTTP 400 — {"errorMessages":["Could not find field: colour"]}<br>Result: **As Expected** | logs/exploratory_session1_todos.log exploratory/session1_todos.sh |
| Step 8 | **Test API POST request**<br>/todos body: {bad json (malformed JSON)<br>Documented? DOCUMENTED<br>Expected: 400 with MalformedJsonException message | **Status 400: Bad Request**<br>Observed: HTTP 400 — {"errorMessages":["com.google.gson.stream.MalformedJsonException: Expected \u0027:\u0027 at line 1 column 7 path $."]}<br>Result: **As Expected**<br>Side note: Malformed-payload probe | logs/exploratory_session1_todos.log exploratory/session1_todos.sh |
| Step 9 | **Test API POST request**<br>/todos body: <todo><title>from xml</title></todo><br>Documented? DOCUMENTED<br>Expected: 201 created from an XML payload | **Status 201: Created**<br>Observed: HTTP 201 — <todo><doneStatus>false</doneStatus><description/><id>404</id><title>from xml</title></todo><br>Result: **As Expected**<br>Side note: Created id is 404: earlier failed creations consumed ids (BUG-07)<br>Format: XML request and response; log evidence | logs/exploratory_session1_todos.log exploratory/session1_todos.sh |
| Step 10 | **Test API POST request**<br>/todos body: <todo><title>broken</todo> (malformed XML)<br>Documented? DOCUMENTED<br>Expected: 400 with mismatched-tag error | **Status 400: Bad Request**<br>Observed: HTTP 400 — {"errorMessages":["Mismatched title and todo at 25 [character 26 line 1]"]}<br>Result: **As Expected**<br>Side note: Malformed-payload probe<br>Format: XML request; log evidence | logs/exploratory_session1_todos.log exploratory/session1_todos.sh |
| Step 11 | **Test API GET request**<br>/todos/1 (Accept: application/xml)<br>Documented? DOCUMENTED<br>Expected: 200 with todo 1 returned as XML | **Status 200: OK**<br>Observed: HTTP 200 — <todos><todo><doneStatus>false</doneStatus><description/><tasksof><id>1</id></tasksof><id>1</id><categories><id>1</id></categories><title>scan paperwork</title></todo></todos><br>Result: **As Expected**<br>Format: XML response; log evidence | logs/exploratory_session1_todos.log exploratory/session1_todos.sh |
| Step 12 | **Test API PUT request**<br>/todos/1 body: {"description":"only description"}<br>Documented? DOCUMENTED<br>Expected: Swagger: PUT amends with the fields in the body, so 200 | **Status 400: Bad Request**<br>Observed: HTTP 400 — {"errorMessages":["title : field is mandatory"]}<br>Result: **Bug**<br>Side note: BUG-05: 400 because title is absent | logs/exploratory_session1_todos.log exploratory/session1_todos.sh |
| Step 13 | **Test API PUT request**<br>/todos/1 body: {"title":"scan paperwork","doneStatus":false}<br>Documented? DOCUMENTED<br>Expected: 200; PUT replaces: omitted description is reset to "" | **Status 200: OK**<br>Observed: HTTP 200 — {"id":"1","title":"scan paperwork","doneStatus":"false","description":""}<br>Result: **As Expected**<br>Side note: PUT replace semantics (contrast POST amend) | logs/exploratory_session1_todos.log exploratory/session1_todos.sh |
| Step 14 | **Test API POST request**<br>/todos/1 body: {"description":"amended"}<br>Documented? DOCUMENTED<br>Expected: 200; POST amends only the given fields | **Status 200: OK**<br>Observed: HTTP 200 — {"id":"1","title":"scan paperwork","doneStatus":"false","description":"amended"}<br>Result: **As Expected** | logs/exploratory_session1_todos.log exploratory/session1_todos.sh |
| Step 15 | **Test API POST request**<br>/todos/1 body: {"description":""}<br>Documented? DOCUMENTED<br>Expected: 200; description amended to empty | **Status 200: OK**<br>Observed: HTTP 200 — {"id":"1","title":"scan paperwork","doneStatus":"false","description":""}<br>Result: **As Expected** | logs/exploratory_session1_todos.log exploratory/session1_todos.sh |

| Time | Endpoint Tested | Response | Reference File |
|---|---|---|---|
| Step 16 | **Test API GET request**<br>/todos?doneStatus=false<br>Documented? UNDOCUMENTED<br>Expected: Filtering is not described in the guide; a filtered list is the sensible behaviour | **Status 200: OK**<br>Observed: HTTP 200 — {"todos":[{"id":"404","title":"from xml","doneStatus":"false","description":""},{"id":"2","title":"file paperwork","doneStatus":"false","description":"","tasksof":[{"id":"1"}]},{"id":"1","title":"scan paperwork","doneStatus":"fals<br>Result: **As Expected**<br>Side note: Undocumented capability: field filtering | logs/exploratory_session1_todos.log<br>exploratory/session1_todos.sh |
| Step 17 | **Test API GET request**<br>/todos?nofield=1<br>Documented? UNDOCUMENTED<br>Expected: Unknown query fields are undefined behaviour | **Status 200: OK**<br>Observed: HTTP 200 — {"todos":[{"id":"400","title":"full","doneStatus":"true","description":"all fields"},{"id":"404","title":"from xml","doneStatus":"false","description":""},{"id":"2","title":"file paperwork","doneStatus":"false","description":"","t<br>Result: **To Explore**<br>Side note: Silently ignored: full list returned, no error | logs/exploratory_session1_todos.log<br>exploratory/session1_todos.sh |
| Step 18 | **Test API GET request**<br>/todos/987654<br>Documented? DOCUMENTED<br>Expected: 404 "could not find an instance" | **Status 404: Not Found**<br>Observed: HTTP 404 — {"errorMessages":["Could not find an instance with todos/987654"]}<br>Result: **As Expected** | logs/exploratory_session1_todos.log<br>exploratory/session1_todos.sh |
| Step 19 | **Test API DELETE request**<br>/todos/987654<br>Documented? DOCUMENTED<br>Expected: 404 for a missing todo | **Status 404: Not Found**<br>Observed: HTTP 404 — {"errorMessages":["Could not find any instances with todos/987654"]}<br>Result: **As Expected**<br>Side note: Wording differs from GET ("any instances") | logs/exploratory_session1_todos.log<br>exploratory/session1_todos.sh |
| Step 20 | **Test API PATCH request**<br>/todos/1<br>Documented? UNDOCUMENTED<br>Expected: Not allowed: 405 | **Status 405: Method Not Allowed**<br>Observed: HTTP 405 — (empty response body)<br>Result: **As Expected** | logs/exploratory_session1_todos.log<br>exploratory/session1_todos.sh |
| Step 21 | **Test API HEAD request**<br>/todos<br>Documented? UNDOCUMENTED<br>Expected: Headers only, status 200 (listed in swagger, not in the guide) | **Status 200: OK**<br>Observed: HTTP 200 — HTTP/1.1 200 OK Date: Wed, 14 Oct 2026 19:08:17 GMT Content-Type: application/json Transfer-Encoding: chunked Server: Jetty(9.4.z-SNAPSHOT)<br>Result: **As Expected** | logs/exploratory_session1_todos.log<br>exploratory/session1_todos.sh |
| Step 22 | **Test API OPTIONS request**<br>/todos<br>Documented? UNDOCUMENTED<br>Expected: 200 with Allow header listing permitted methods | **Status 200: OK**<br>Observed: HTTP 200 — HTTP/1.1 200 OK Date: Wed, 14 Oct 2026 19:08:18 GMT Allow: OPTIONS, GET, HEAD, POST Content-Type: text/html;charset=utf-8 Transfer-Encoding: chunked Server: Jetty(9.4.z-SNAPSHOT)<br>Result: **As Expected** | logs/exploratory_session1_todos.log<br>exploratory/session1_todos.sh |
| Close | **Review findings and cross-reference bugs** | Findings, concerns, risks and new test ideas reviewed and recorded in Section 4; bugs entered in bug_reports.md with the same BUG numbers used there. | bug_reports.md |

## Section 3: Referenced Files (Scripts, Programs, Screen Shots, Video Clips, Spreadsheets, etc)

Table 3: Descriptions of Referenced Files and Created Scripts

| File Name | Description |
|---|---|
| exploratory/session1_todos.sh | Curl capability script used in this session: demonstrates every capability exercised, in order. Shared helper: exploratory/common.sh. |
| logs/exploratory_session1_todos.log | Raw session log: the exact status code and response body of every request in Table 2, in step order. This log is the evidence for the Response column (no screenshots were taken; the log is replayable via the script). |
| session_notes/CH1_Todos_Notes.pdf | This session-notes document. The teacher-template data is contained in this PDF itself: participant/charter information in Section 1, the per-request method, method type, documented status, expected behaviour, expected-vs-observed verdict, observed behaviour, side notes and XML/evidence details in Section 2, and the summary of findings, concerns and test ideas in Section 4. |
| bug_reports.md | Consolidated bug form and the full bug reports (summary, description, impact, steps to reproduce) for the bugs in Table 4. |
| tests/test_todos_api.py,<br>tests/test_todo_relationships_api.py,<br>tests/test_payload_formats_api.py,<br>tests/test_bugs_expected_behavior.py,<br>tests/test_bugs_actual_behavior.py | Unit tests created from this session's findings, including the expected-behaviour / actual-behaviour pairs for each numbered bug. |
| Part_A_Report.pdf | Final Part A report: deliverables, findings, suite structure and repository description. |

## Section 4: Summary of Session Findings

### 4.1 Summary of Session Findings

Capabilities identified:

- CRUD on todos: GET list, GET :id, POST create (201), POST :id amend (200), PUT :id replace (200), DELETE :id (200) — documented.
- Field defaults on create: doneStatus=false, description="".
- Query filtering ?title= and ?doneStatus= — undocumented.
- XML input and output via Content-Type / Accept — documented in /docs.
- HEAD and OPTIONS on the collection — undocumented in the guide (listed in swagger).

What was tried:

Create with title only, all fields, no title, empty title, bad boolean, client-set id, unknown field, malformed JSON, malformed XML. Amend via POST vs PUT; PUT with description only; GET/DELETE/PATCH/HEAD/OPTIONS on existing and missing ids; filtering by known and unknown fields.

What we learned:

- Todo CRUD works as documented; validation is strict and error messages are clear and consistent.
- PUT without title returns 400 although swagger describes PUT as amending with the fields in the body (BUG-05).
- doneStatus is returned as a string ("false") but a string is refused on input (BUG-06).
- Failed creations still consume ids (BUG-07); unknown filter fields are silently ignored.
- A 10,000-character title is accepted; a JSON body is accepted even when Content-Type claims XML (body is sniffed).

Decisions made:

- Treat every swagger statement as the oracle; treat undocumented-but-sensible behaviour as "actual behaviour" tests rather than bugs.
- Probe with typical data first, then boundaries (empty / long / unicode / malformed).

Table 4: Descriptions of Identified Bugs

| Bug | Description |
|---|---|
| BUG-05 — PUT /todos/{id} with only a description in the body | Swagger describes PUT as amending with the fields in the body, but the request is rejected: 400 “title : field is mandatory”. Clients following the documentation cannot perform a partial PUT update. |
| BUG-06 — doneStatus returned as a string, rejected as a string | Responses contain "doneStatus":"false" (a string), yet sending that same string back returns 400 “doneStatus should be BOOLEAN”. Round-tripping a GET response into a POST/PUT fails. |
| BUG-07 — Failed creations still consume ids | Each rejected POST /todos advances the id counter (in this session the first successful XML create received id 404 after the failed attempts). Ids develop gaps and leak how many failed requests occurred. |

### 4.2 Concerns Identified in Session / Areas of Potential Risk

- BUG-05, BUG-06, BUG-07; unbounded title length; inconsistent error wording between GET and DELETE on a missing id.
- No length limit on title (memory / DoS exposure).
- Failed requests burn ids, so id sequences leak request volume and cannot be restored.
- String-for-boolean responses break round-tripping clients.

### 4.3 New Testing Ideas Identified in Session

- Fuzz doneStatus ("TRUE", 1, null).
- Very large request bodies.
- Concurrent POSTs racing for ids.
- Titles with control characters.
- Filter combinations (title + doneStatus).